

Faculty of Engineering - Alexandria University

Computer and Communications Engineering

Lab 4 - Weather Stations Monitoring

Technical Report

Distributed IoT Weather Monitoring

10 Weather Stations → Apache Kafka → Rain Processor
→ Central Station → PostgreSQL

Course: Net Centric Computing
Academic Year: Fall 2025–2026
Date: August 24, 2026

Team Members

Name	ID
Ahmed Walaaeldin	8625
Mazen Adel	8634
Marwan Ahmed	8824
Ahmed Ibrahim	8672

Contents

1	Introduction	4
2	System Architecture	4
2.1	High-Level Design	4
2.2	Repository Organization	5
3	Weather Station Mock - Part A	6
3.1	Reading Generation	6
3.2	Battery Status Distribution	6
3.3	Simulated Weather and Message Drops	7
4	Kafka Integration - Part B	7
4.1	Producer Configuration	7
4.2	Partitioning and Ordering	8
5	Rain Detection - Part C	8
5.1	Kafka Streams Processor API	8
5.2	Processor API Versus DSL	9
6	Central Station - Part D	9
6.1	Concurrent Consumers	10
6.2	Batch Persistence	10
6.3	Delivery Semantics and Idempotency	10
6.4	Database Schema	10
7	Historical Weather Analysis - Part E	11
7.1	Battery Status Distribution Per Station	11
7.2	Dropped Messages Per Station	12
7.3	Latest Weather Status Per Station	13
7.4	Rain Alert Cross-Check	13
8	Kubernetes Deployment - Part F	14
8.1	Containerization	14
8.2	Kubernetes Resources	14
8.3	Why a StatefulSet is Used for Weather Stations	15
8.4	Database Persistence	15
8.5	Configuration and Secrets	15
8.6	Runtime Evidence	16
9	Cloud Deployment - Bonus	18
9.1	Target Topology	18
9.2	Machine and Provider Details	18
9.3	Network Configuration	18
9.4	Deployment Procedure	18
10	Challenges and Lessons Learned	19

11 Conclusion

20

List of Figures

1	Implemented distributed weather monitoring architecture.	4
2	Project repository structure.	5
3	Weather Station 1 logs showing periodic sends and simulated message drops. .	7
4	Kafka console consumer showing messages from the weather-readings topic.	8
5	Kafka topics created for weather readings and rain alerts.	8
6	Rain processor logs showing humidity values above 70% being detected. .	9
7	Kafka console consumer showing messages from the rain-alerts topic. .	9
8	Central station logs showing Kafka consumption and database persistence activity.	11
9	Measured battery status distribution per station.	12
10	Dropped-message calculation per weather station.	13
11	Latest weather status queried directly from PostgreSQL for each station. .	13
12	Cross-check between raw humidity readings and stored rain alerts.	14
13	Kubernetes pod status after deploying the complete weather monitoring system.	15
14	Kubernetes Services and the Weather Station StatefulSet.	16
15	Kubernetes ConfigMap and Secret configuration.	16
16	Weather Station pod logs from weather-station-0	17
17	Rain Processor running in Kubernetes and detecting humidity greater than 70%.	17
18	Central Station logs showing batch insert activity inside Kubernetes. . .	17
19	Database query results executed against the Kubernetes deployment. . .	17
20	Bonus cloud deployment topology.	18

1 Introduction

The objective of this lab is to design and implement a distributed weather monitoring system that models a practical Internet of Things (IoT) stream-processing pipeline. Multiple weather stations continuously generate readings, Apache Kafka provides asynchronous transport and buffering, a Kafka Streams processor detects special weather conditions, and a central station persists historical data to a relational SQL database.

The implemented system follows the required architecture while adding reliability and deployment considerations. Ten independent weather station services emit one reading per second. Each generated reading contains a station identifier, a per-station sequence number, battery status, timestamp, humidity, temperature, and wind speed. Approximately 10% of generated messages are intentionally dropped to simulate unreliable sensing or transmission. The battery status is sampled to converge statistically toward a 30% low, 40% medium, and 30% high distribution.

Apache Kafka decouples producers from downstream processing. Weather readings are published to the `weather-readings` topic, while a Kafka Streams processor identifies readings with humidity greater than 70% and publishes a special alert to `rain-alerts`. The central station consumes the streams and writes them to PostgreSQL. Weather readings are inserted using JDBC batches to reduce database I/O, and the application commits Kafka offsets only after the corresponding database transaction succeeds.

The system was first validated locally, then deployed on Kubernetes using Minikube. The bonus deployment separates weather stations from the central base station across cloud virtual machines and uses an externally managed PostgreSQL database.

2 System Architecture

2.1 High-Level Design

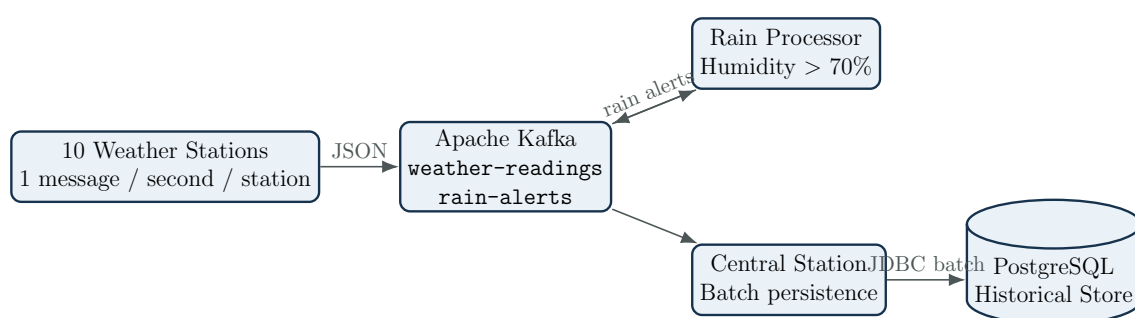


Figure 1: Implemented distributed weather monitoring architecture.

The architecture is divided into three main stages:

1. **Data acquisition:** ten weather station services generate simulated sensor readings and publish them to Kafka.
2. **Data processing:** Kafka buffers the readings, while the rain processor filters and transforms readings with humidity above the configured threshold.

3. **Data persistence and analysis:** the central station stores readings and rain alerts in PostgreSQL, from which historical validation queries are executed.

Kafka is placed between acquisition and processing to provide loose coupling, buffering, replay capability, partition-based ordering, and independent scaling of producers and consumers.

2.2 Repository Organization

The implementation is structured as a Maven multi-module project. The `common` module provides shared message classes and configuration helpers. Separate modules implement the weather stations, rain processor, and central station. Deployment assets are organized into Docker, Kubernetes, SQL, and cloud-specific directories.

Everything is working correctly. Here's what each screenshot proves against the lab requirements:

Screenshot	Lab Part	What it proves	✓ / ✗
1 - docker ps	F	10 stations + kafka + postgres + rain-processor + central-station all running	✓
2 - station logs	A+B	1 msg/sec, correct JSON schema, sent/dropped counting, battery values	✓
3 - rain-processor logs	C	Detects humidity >70%, all 10 stations, running alert counter	✓
4 - central-station logs	D	Batch flushes (total 9568), rain alerts persisted	✓
5 - kafka weather-readings	B	Correct JSON schema on the topic, multiple stations visible	✓
6 - kafka rain-alerts	C	"It is raining" messages, humidity >70%, alert_count field	✓
7 - battery distribution	E-1	All hovering around 30/40/30 across 10 stations	✓
8 - dropped messages	E-2	Drop rates 8.97%-11.83%, averaging ~10%	✓
9 - latest status	D	Latest reading per station from the DB (10 rows)	✓
10 - rain cross-check	C+D	3770 readings >70% ≈ 3798 alerts stored (29.6% of total)	✓
11 - kafka topics	B+C	<code>weather-readings</code> , <code>rain-alerts</code> , and the state store changelog topic	✓

Figure 2: Project repository structure.

Table 1: Main repository components.

Path	Purpose
<code>common/</code>	Shared JSON models, Kafka topic names, and environment-based configuration helpers.
<code>weather-station/</code>	Parts A and B: reading generation and Kafka production.
<code>rain-processor/</code>	Part C: Kafka Streams rain detection using the Processor API, plus a DSL alternative.
<code>central-station/</code>	Part D: Kafka consumers, JDBC batching, and database initialization.
<code>sql/</code>	Part E: relational schema and historical analysis queries.
<code>k8s/</code>	Part F: Kubernetes manifests and deployment script.
<code>cloud/</code>	Bonus: split two-VM deployment and managed PostgreSQL configuration.

3 Weather Station Mock - Part A

3.1 Reading Generation

Each weather station generates one `WeatherMessage` every second. The station identity is either supplied explicitly using the `STATION_ID` environment variable or derived from the ordinal of a Kubernetes StatefulSet pod. This allows pod names `weather-station-0` through `weather-station-9` to map automatically to logical station IDs 1 through 10.

The generated message schema is:

Listing 1: Weather status JSON schema.

```

1 {
2   "station_id": 1,
3   "s_no": 1,
4   "battery_status": "low",
5   "status_timestamp": 1681521224,
6   "weather": {
7     "humidity": 35,
8     "temperature": 100,
9     "wind_speed": 13
10  }
11 }
```

The sequence number is incremented for every generated sample *before* the drop decision. Therefore, intentionally dropped messages leave observable gaps in the stored sequence numbers. This design enables Part E to estimate the drop rate directly from the database.

3.2 Battery Status Distribution

A single uniform random value in the range $[0, 1)$ is used to choose the battery state:

- `low` when $p < 0.30$,

- medium when $0.30 \leq p < 0.70$,
- high otherwise.

This produces the required theoretical distribution of 30% / 40% / 30%, with small deviations expected for finite sample sizes.

3.3 Simulated Weather and Message Drops

Humidity is generated uniformly between 0 and 100%, temperature between 20 and 120 degrees Fahrenheit, and wind speed between 0 and 50 km/h. A second random draw decides whether each generated message is intentionally dropped with probability 0.10. Even if a message is dropped, its sequence number is not reused.

```
(base) ahmedwale@hmds-air distributed-weather-monitoring-system % docker compose logs --tail=30 weather-station
weather-station-1 | [station-1] sent=1928 dropped=139 last=(station_id:1,"s_no":1159,"battery_status":"medium","status_timestamp":1787493422,"weather":{"humidity":24,"temperature":45,"wind_speed":12}}
weather-station-1 | [station-1] dropped_s_no=1171 (total dropped=140)
weather-station-1 | [station-1] sent=1888 dropped=141 last=(station_id:1,"s_no":1191,"battery_status":"medium","status_timestamp":1787493454,"weather":{"humidity":61,"temperature":119,"wind_speed":39}}
weather-station-1 | [station-1] sent=1888 dropped=145 last=(station_id:1,"s_no":1225,"battery_status":"high","status_timestamp":1787493488,"weather":{"humidity":59,"temperature":58,"wind_speed":15}}
weather-station-1 | [station-1] sent=1118 dropped=147 last=(station_id:1,"s_no":1257,"battery_status":"low","status_timestamp":1787493520,"weather":{"humidity":89,"temperature":61,"wind_speed":28}}
weather-station-1 | [station-1] dropped_s_no=1271 (total dropped=158)
weather-station-1 | [station-1] sent=1148 dropped=153 last=(station_id:1,"s_no":1293,"battery_status":"medium","status_timestamp":1787493556,"weather":{"humidity":61,"temperature":88,"wind_speed":24}}
weather-station-1 | [station-1] sent=1178 dropped=157 last=(station_id:1,"s_no":1327,"battery_status":"medium","status_timestamp":1787493590,"weather":{"humidity":26,"temperature":87,"wind_speed":12}}
weather-station-1 | [station-1] sent=1288 dropped=158 last=(station_id:1,"s_no":1358,"battery_status":"medium","status_timestamp":1787493621,"weather":{"humidity":40,"temperature":63,"wind_speed":44}}
weather-station-1 | [station-1] dropped_s_no=1372 (total dropped=168)
weather-station-1 | [station-1] sent=1238 dropped=168 last=(station_id:1,"s_no":1398,"battery_status":"high","status_timestamp":1787493653,"weather":{"humidity":60,"temperature":31,"wind_speed":29}}
weather-station-1 | [station-1] sent=1268 dropped=161 last=(station_id:1,"s_no":1421,"battery_status":"high","status_timestamp":1787493684,"weather":{"humidity":20,"temperature":48,"wind_speed":10}}
weather-station-1 | [station-1] sent=1298 dropped=162 last=(station_id:1,"s_no":1452,"battery_status":"low","status_timestamp":1787493715,"weather":{"humidity":90,"temperature":97,"wind_speed":41}}
weather-station-1 | [station-1] sent=1328 dropped=166 last=(station_id:1,"s_no":1486,"battery_status":"low","status_timestamp":1787493749,"weather":{"humidity":29,"temperature":35,"wind_speed":8}}
weather-station-1 | [station-1] dropped_s_no=1513 (total dropped=178)
weather-station-1 | [station-1] sent=1358 dropped=178 last=(station_id:1,"s_no":1520,"battery_status":"high","status_timestamp":1787493783,"weather":{"humidity":73,"temperature":56,"wind_speed":13}}
weather-station-1 | [station-1] sent=1388 dropped=177 last=(station_id:1,"s_no":1557,"battery_status":"low","status_timestamp":1787493820,"weather":{"humidity":32,"temperature":91,"wind_speed":8}}
weather-station-1 | [station-1] sent=1418 dropped=178 last=(station_id:1,"s_no":1588,"battery_status":"low","status_timestamp":1787493851,"weather":{"humidity":29,"temperature":37,"wind_speed":3}}
weather-station-1 | [station-1] dropped_s_no=1619 (total dropped=188)
weather-station-1 | [station-1] sent=1448 dropped=188 last=(station_id:1,"s_no":1620,"battery_status":"medium","status_timestamp":1787493883,"weather":{"humidity":17,"temperature":28,"wind_speed":42}}
weather-station-1 | [station-1] sent=1478 dropped=182 last=(station_id:1,"s_no":1652,"battery_status":"high","status_timestamp":1787493915,"weather":{"humidity":68,"temperature":101,"wind_speed":3}}
weather-station-1 | [station-1] sent=1508 dropped=187 last=(station_id:1,"s_no":1687,"battery_status":"medium","status_timestamp":1787493950,"weather":{"humidity":48,"temperature":102,"wind_speed":7}}
weather-station-1 | [station-1] sent=1538 dropped=189 last=(station_id:1,"s_no":1719,"battery_status":"medium","status_timestamp":1787493983,"weather":{"humidity":85,"temperature":28,"wind_speed":16}}
weather-station-1 | [station-1] dropped_s_no=1731 (total dropped=198)
weather-station-1 | [station-1] sent=1568 dropped=193 last=(station_id:1,"s_no":1753,"battery_status":"medium","status_timestamp":1787494017,"weather":{"humidity":5,"temperature":48,"wind_speed":37}}
weather-station-1 | [station-1] dropped_s_no=1789 (total dropped=208)
weather-station-1 | [station-1] sent=1598 dropped=208 last=(station_id:1,"s_no":1790,"battery_status":"high","status_timestamp":1787494054,"weather":{"humidity":4,"temperature":91,"wind_speed":10}}
weather-station-1 | [station-1] sent=1628 dropped=203 last=(station_id:1,"s_no":1823,"battery_status":"high","status_timestamp":1787494087,"weather":{"humidity":33,"temperature":102,"wind_speed":2}}
weather-station-1 | [station-1] sent=1658 dropped=205 last=(station_id:1,"s_no":1855,"battery_status":"high","status_timestamp":1787494119,"weather":{"humidity":80,"temperature":66,"wind_speed":33}}
weather-station-1 | [station-1] sent=1688 dropped=208 last=(station_id:1,"s_no":1888,"battery_status":"medium","status_timestamp":1787494152,"weather":{"humidity":9,"temperature":44,"wind_speed":18}}
(base) ahmedwale@hmds-air distributed-weather-monitoring-system %
```

Figure 3: Weather Station 1 logs showing periodic sends and simulated message drops.

4 Kafka Integration - Part B

4.1 Producer Configuration

The weather station uses the Java Kafka Producer API to publish serialized JSON to the `weather-readings` topic. The key configuration choices are summarized below.

Table 2: Weather station Kafka producer configuration.

Property	Value	Reason
Bootstrap servers	Environment configured	Supports local, Kubernetes, and cloud deployments.
Key serializer	StringSerializer	Station ID is used as the message key.
Value serializer	StringSerializer	Weather messages are serialized to JSON strings.
acks	all	Improves durability by requiring broker acknowledgement.
Idempotence	enabled	Prevents duplicate writes caused by producer retries.
Retries	3	Handles transient Kafka communication failures.
linger.ms	20 ms	Allows small batching without significantly affecting one-second cadence.
Compression	gzip	Reduces network payload size.

4.2 Partitioning and Ordering

The Kafka record key is the station ID. Kafka therefore routes all readings from one station consistently to the same partition. This preserves the relative order of messages for each individual station while still allowing records from different stations to be distributed across multiple partitions.

The deployment creates ten partitions for `weather-readings`, matching the ten logical stations and providing a clear path for parallel downstream processing.

```
(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=kafka) \
/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic weather-readings --max-messages 5
The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out. See https://kafka.apache.org/documentation/#consumer_rebalance_protocol
{"station_id":1,"s_no":2078,"battery_status":"medium","status_timestamp":1787494342,"weather":{"humidity":62,"temperature":63,"wind_speed":47}}
{"station_id":9,"s_no":1021,"battery_status":"high","status_timestamp":1787494342,"weather":{"humidity":6,"temperature":87,"wind_speed":34}}
{"station_id":3,"s_no":1027,"battery_status":"medium","status_timestamp":1787494342,"weather":{"humidity":47,"temperature":54,"wind_speed":37}}
{"station_id":16,"s_no":1025,"battery_status":"medium","status_timestamp":1787494343,"weather":{"humidity":11,"temperature":144,"wind_speed":30}}
{"station_id":7,"s_no":1024,"battery_status":"medium","status_timestamp":1787494343,"weather":{"humidity":59,"temperature":76,"wind_speed":11}}
Processed a total of 5 messages
```

Figure 4: Kafka console consumer showing messages from the `weather-readings` topic.

```
(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=kafka) \
/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
__consumer_offsets
rain-alerts
rain-detector-app-rain-alert-counts-changelog
weather-readings
```

Figure 5: Kafka topics created for weather readings and rain alerts.

5 Rain Detection - Part C

5.1 Kafka Streams Processor API

The primary rain detection implementation uses the low-level Kafka Streams Processor API. Its topology is conceptually:

weather-readings → RainProcessor → rain-alerts

Each incoming record is deserialized into a `WeatherMessage`. Malformed records are skipped safely. If the weather object is absent or the humidity is less than or equal to 70%, no downstream record is produced. For readings with humidity greater than 70%, the processor increments a per-station counter stored in a persistent key-value state store and forwards a `RainAlert`.

A rain alert contains the original station ID, sequence number, timestamp, humidity, cumulative per-station alert count, and the message "It is raining".

5.2 Processor API Versus DSL

The project also contains an alternative Kafka Streams DSL implementation. The DSL version is concise and expresses the transformation as a stream of `mapValues`, `filter`, and `to` operations. The Processor API version was selected as the primary implementation because it provides explicit control over processor state, state-store access, and record forwarding. This makes the stateful per-station alert counter straightforward to demonstrate.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker compose logs --tail=30 rain-processor
rain-processor-1 | [rain-processor] RAIN station=4 s_no=917 humidity=72% (alert #225)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=912 humidity=87% (alert #232)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=916 humidity=83% (alert #237)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=920 humidity=84% (alert #240)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=919 humidity=91% (alert #226)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=918 humidity=98% (alert #238)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=920 humidity=88% (alert #237)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=923 humidity=97% (alert #241)
rain-processor-1 | [rain-processor] RAIN station=1 s_no=1974 humidity=98% (alert #498)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=920 humidity=95% (alert #239)
rain-processor-1 | [rain-processor] RAIN station=5 s_no=922 humidity=96% (alert #264)
rain-processor-1 | [rain-processor] RAIN station=3 s_no=924 humidity=77% (alert #248)
rain-processor-1 | [rain-processor] RAIN station=9 s_no=918 humidity=76% (alert #241)
rain-processor-1 | [rain-processor] RAIN station=1 s_no=1975 humidity=84% (alert #499)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=922 humidity=87% (alert #238)
rain-processor-1 | [rain-processor] RAIN station=8 s_no=920 humidity=98% (alert #256)
rain-processor-1 | [rain-processor] RAIN station=7 s_no=922 humidity=82% (alert #240)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=923 humidity=72% (alert #239)
rain-processor-1 | [rain-processor] RAIN station=8 s_no=922 humidity=84% (alert #257)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=927 humidity=87% (alert #242)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=920 humidity=89% (alert #233)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=926 humidity=85% (alert #227)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=925 humidity=88% (alert #240)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=921 humidity=85% (alert #234)
rain-processor-1 | [rain-processor] RAIN station=6 s_no=926 humidity=79% (alert #241)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=928 humidity=78% (alert #228)
rain-processor-1 | [rain-processor] RAIN station=2 s_no=930 humidity=73% (alert #243)
rain-processor-1 | [rain-processor] RAIN station=4 s_no=929 humidity=94% (alert #229)
rain-processor-1 | [rain-processor] RAIN station=3 s_no=930 humidity=73% (alert #249)
rain-processor-1 | [rain-processor] RAIN station=10 s_no=924 humidity=91% (alert #235)
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 6: Rain processor logs showing humidity values above 70% being detected.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=kafka) \
/opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
--topic rain-alerts --max-messages 5
The consumer rebalance protocol (KIP-248) is production-ready! Set group.protocol=consumer to try it out. See https://kafka.apache.org/documentation/#consumer_rebalance_protocol
{"station_id":14,"s_no":1088,"status_timestamp":1787494485,"humidity":88,"alert_count":274,"message":"It is raining"}
{"station_id":3,"s_no":1089,"status_timestamp":1787494485,"humidity":81,"alert_count":297,"message":"It is raining"}
{"station_id":7,"s_no":1086,"status_timestamp":1787494485,"humidity":85,"alert_count":282,"message":"It is raining"}
{"station_id":6,"s_no":1087,"status_timestamp":1787494485,"humidity":76,"alert_count":281,"message":"It is raining"}
{"station_id":12,"s_no":1090,"status_timestamp":1787494485,"humidity":77,"alert_count":292,"message":"It is raining"}
Processed a total of 5 messages
```

Figure 7: Kafka console consumer showing messages from the rain-alerts topic.

6 Central Station - Part D

6.1 Concurrent Consumers

The central station initializes the database schema and runs two Kafka consumers in separate threads:

- a weather readings consumer for `weather-readings`;
- a rain alerts consumer for `rain-alerts`.

Separate JDBC connections are used because JDBC connections are not thread-safe. Both consumers use manual Kafka offset commits.

6.2 Batch Persistence

The weather readings consumer buffers messages in memory and flushes them to PostgreSQL when either:

- the buffer reaches the configured batch size of 5000 records, or
- the flush interval of 10 seconds expires while pending records exist.

The batch writer uses a prepared statement and JDBC `executeBatch()` so a group of readings can be persisted in one transaction rather than requiring one network and transaction round trip per row. This significantly reduces database I/O overhead under sustained streaming load.

6.3 Delivery Semantics and Idempotency

The persistence sequence is deliberately:

1. write the batch to PostgreSQL;
2. commit the PostgreSQL transaction;
3. commit Kafka offsets.

This provides at-least-once delivery. If the process fails after the database commit but before the Kafka offset commit, the same Kafka records may be consumed again after restart. To make such reprocessing harmless, the table enforces `UNIQUE(station_id, sequence_number)` and the insert statement uses `ON CONFLICT DO NOTHING`. Consequently, re-read records do not produce duplicate historical rows.

6.4 Database Schema

The main table stores all weather readings and includes indexes optimized for per-station sequence queries and battery distribution analysis. A second table archives rain alerts. The `latest_station_status` view uses PostgreSQL `DISTINCT ON` to return the latest sequence for every station.

Listing 2: Core weather readings schema.

```
1 CREATE TABLE IF NOT EXISTS weather_readings (  
2     id BIGSERIAL PRIMARY KEY,  
3     station_id BIGINT NOT NULL,  
4     sequence_number BIGINT NOT NULL,
```

```

5  battery_status VARCHAR(10) NOT NULL,
6  timestamp BIGINT NOT NULL,
7  humidity INT,
8  temperature INT,
9  wind_speed INT,
10 ingested_at TIMESTAMPTZ NOT NULL DEFAULT now(),
11 CONSTRAINT uq_station_sequence
12     UNIQUE (station_id, sequence_number)
13 );

```

```

[(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker compose logs --tail=30 central-station
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] batch flushed: 94 rows (total 9480)
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 3 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 3 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 2 rain alerts
central-station-1 | [central] batch flushed: 88 rows (total 9568)
central-station-1 | [central] persisted 3 rain alerts
central-station-1 | [central] persisted 1 rain alerts
central-station-1 | [central] persisted 1 rain alerts
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %

```

Figure 8: Central station logs showing Kafka consumption and database persistence activity.

7 Historical Weather Analysis - Part E

The SQL analysis validates the statistical properties of the mock stations and verifies that the stream processor and database remain consistent.

7.1 Battery Status Distribution Per Station

The following analysis groups records by station and battery status, then calculates the percentage of each battery class. With a sufficiently large sample, each station is expected to approach 30% low, 40% medium, and 30% high.

Listing 3: Battery distribution query.

```

1 SELECT station_id,
2        COUNT(*) AS total,
3        ROUND(100.0 * COUNT(*) FILTER
4            (WHERE battery_status = 'low') / COUNT(*), 2) AS low_pct,
5        ROUND(100.0 * COUNT(*) FILTER

```

```

6         (WHERE battery_status = 'medium') / COUNT(*), 2) AS medium_pct,
7         ROUND(100.0 * COUNT(*) FILTER
8         (WHERE battery_status = 'high') / COUNT(*), 2) AS high_pct
9 FROM weather_readings
10 GROUP BY station_id
11 ORDER BY station_id;

```

```

(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=postgres) \
psql -U weather -d weather -c "
SELECT station_id,
COUNT(*) AS received,
ROUND(100.0*COUNT(*) FILTER (WHERE battery_status='low')/COUNT(*),2) AS low_pct,
ROUND(100.0*COUNT(*) FILTER (WHERE battery_status='medium')/COUNT(*),2) AS med_pct,
ROUND(100.0*COUNT(*) FILTER (WHERE battery_status='high')/COUNT(*),2) AS high_pct
FROM weather_readings GROUP BY station_id ORDER BY station_id;
"

```

station_id	received	low_pct	med_pct	high_pct
1	1929	28.41	41.37	30.22
2	1020	29.31	39.02	31.67
3	995	29.05	43.12	27.84
4	996	30.02	39.16	30.82
5	1009	33.70	38.16	28.15
6	982	30.35	43.08	26.58
7	983	31.23	38.96	29.81
8	986	32.05	36.71	31.24
9	1008	27.98	38.79	33.23
10	1010	29.01	40.20	30.79

(10 rows)

Figure 9: Measured battery status distribution per station.

Minor deviations from the exact 30/40/30 ratio are expected because each message is generated by an independent random draw. As the number of samples increases, the measured distribution converges toward the configured probabilities.

7.2 Dropped Messages Per Station

Because the sequence number increments for every generated sample, the highest stored sequence number approximates the total number of readings that should have existed up to that point. The difference between the maximum sequence and the number of stored rows indicates dropped messages.

Listing 4: Dropped message estimation query.

```

1 SELECT station_id,
2        MAX(sequence_number) AS expected_messages,
3        COUNT(*) AS received_messages,
4        MAX(sequence_number) - COUNT(*) AS dropped_messages,
5        ROUND(100.0 * (MAX(sequence_number) - COUNT(*)) /
6        MAX(sequence_number), 2) AS drop_rate_pct
7 FROM weather_readings
8 GROUP BY station_id
9 ORDER BY station_id;

```



```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=postgres) \
psql -U weather -d weather -c "
SELECT station_id,
       MAX(sequence_number) AS expected,
       COUNT(*) AS received,
       MAX(sequence_number) - COUNT(*) AS dropped,
       ROUND(100.0*(MAX(sequence_number)-COUNT(*)/MAX(sequence_number),2) AS drop_pct
FROM weather_readings GROUP BY station_id ORDER BY station_id;
"

```

station_id	expected	received	dropped	drop_pct
1	2221	1974	247	11.12
2	1171	1066	105	8.97
3	1170	1044	126	10.77
4	1169	1041	128	10.95
5	1169	1056	113	9.67
6	1168	1030	138	11.82
7	1167	1029	138	11.83
8	1166	1034	132	11.32
9	1164	1054	110	9.45
10	1163	1056	107	9.20

(10 rows)

Figure 10: Dropped-message calculation per weather station.

The measured value is expected to approach the configured 10% drop rate. Small differences are normal for finite runs.

7.3 Latest Weather Status Per Station

The `latest_station_status` database view returns one row per station, selecting the greatest observed sequence number for that station.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=postgres) \
psql -U weather -d weather -c "
SELECT station_id, sequence_number, battery_status, humidity, temperature, wind_speed
FROM weather_readings w
WHERE sequence_number = (SELECT MAX(sequence_number) FROM weather_readings w2 WHERE w2.station_id = w.station_id)
ORDER BY station_id;
"

```

station_id	sequence_number	battery_status	humidity	temperature	wind_speed
1	2342	high	88	92	0
2	1292	medium	97	76	50
3	1291	low	100	33	42
4	1290	low	67	39	9
5	1289	low	23	106	29
6	1289	medium	25	57	38
7	1288	low	77	20	32
8	1287	low	0	27	39
9	1285	high	30	90	7
10	1284	medium	74	72	33

(10 rows)

Figure 11: Latest weather status queried directly from PostgreSQL for each station.

7.4 Rain Alert Cross-Check

The validation query compares the count of persisted raw readings with humidity greater than 70% against the number of rain alerts stored for the same station. The two values should match or converge after all consumers have caught up.

```
(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system % docker exec -it $(docker ps -qf name=postgres) \
psql -U weather -d weather -c "
SELECT COUNT(*) AS total_readings,
COUNT(*) FILTER (WHERE humidity > 70) AS readings_above_70,
(SELECT COUNT(*) FROM rain_alerts) AS rain_alerts_stored
FROM weather_readings;
"
total_readings | readings_above_70 | rain_alerts_stored
-----
12753 | 3770 | 3798
(1 row)
```

Figure 12: Cross-check between raw humidity readings and stored rain alerts.

8 Kubernetes Deployment - Part F

8.1 Containerization

The Weather Station, Rain Processor, and Central Station each use a multi-stage Dockerfile. Maven with Eclipse Temurin JDK 17 is used during the build stage, while the runtime stage contains only Eclipse Temurin JRE 17 and the shaded application JAR. Multi-stage builds keep the final runtime images smaller and avoid including Maven tooling in production containers.

8.2 Kubernetes Resources

The Kubernetes deployment is organized into separate manifests for the namespace and shared configuration, Zookeeper, Kafka, PostgreSQL, weather stations, rain processor, and central station.

Table 3: Kubernetes resource inventory.

Resource	Role
Namespace weather	Isolates all lab resources.
ConfigMap weather-config	Stores non-sensitive runtime configuration such as Kafka endpoints, topics, timing, drop rate, batch size, and DB URL.
Secret db-credentials	Supplies database username and password to the relevant pods.
Zookeeper Deployment	Coordination service for the selected Kafka configuration.
Kafka Deployment + Service	Broker reachable through cluster DNS at kafka:9092 .
Kafka init Job	Explicitly creates weather-readings and rain-alerts with 10 partitions.
PostgreSQL Deployment	Relational historical store.
PV + PVC	Persists database files independently of the PostgreSQL pod lifecycle.
Weather Station StatefulSet	Runs 10 replicas with stable ordinal identities.
Rain Processor Deployment	Runs the Kafka Streams processing application.
Central Station Deployment	Consumes topics and writes to PostgreSQL.

8.3 Why a StatefulSet is Used for Weather Stations

A StatefulSet gives every station pod a stable ordinal name. The application parses that ordinal and maps it to station IDs 1 through 10, avoiding ten separate manifests and ensuring each simulated station has a stable identity. The StatefulSet uses parallel pod management so the ten mock stations can start concurrently.

8.4 Database Persistence

PostgreSQL mounts a 5 GiB PersistentVolumeClaim. The PersistentVolume uses a **Retain** reclaim policy, so deleting or restarting the PostgreSQL pod does not automatically remove the underlying stored weather history. This separates compute lifecycle from database storage lifecycle during the lab deployment.

8.5 Configuration and Secrets

General runtime values are supplied through a ConfigMap, while database credentials are provided using a Kubernetes Secret and injected as environment variables. The application itself does not hardcode database credentials.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP              NODE                                NOMINATED NODE   READINESS GATES
central-station-845c95bb9-47s79    1/1     Running   0           67s   10.244.0.20     desktop-control-plane              <none>            <none>
kafka-6ff674c684-5hn8h             1/1     Running   0           17m   10.244.0.5      desktop-control-plane              <none>            <none>
postgres-58d5565c9-g2qnd           1/1     Running   0           16m   10.244.0.6      desktop-control-plane              <none>            <none>
rain-processor-597964d56d-tf7pr     1/1     Running   0           68s   10.244.0.19     desktop-control-plane              <none>            <none>
weather-station-0                   1/1     Running   0           67s   10.244.0.22     desktop-control-plane              <none>            <none>
weather-station-1                   1/1     Running   0           67s   10.244.0.21     desktop-control-plane              <none>            <none>
weather-station-2                   1/1     Running   0           67s   10.244.0.23     desktop-control-plane              <none>            <none>
weather-station-3                   1/1     Running   0           67s   10.244.0.25     desktop-control-plane              <none>            <none>
weather-station-4                   1/1     Running   0           67s   10.244.0.24     desktop-control-plane              <none>            <none>
weather-station-5                   1/1     Running   0           67s   10.244.0.28     desktop-control-plane              <none>            <none>
weather-station-6                   1/1     Running   0           67s   10.244.0.26     desktop-control-plane              <none>            <none>
weather-station-7                   1/1     Running   0           67s   10.244.0.27     desktop-control-plane              <none>            <none>
weather-station-8                   1/1     Running   0           67s   10.244.0.30     desktop-control-plane              <none>            <none>
weather-station-9                   1/1     Running   0           67s   10.244.0.29     desktop-control-plane              <none>            <none>
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 13: Kubernetes pod status after deploying the complete weather monitoring system.


```

(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/central-station-845c95bb9-47s79 1/1     Running   0           3h17m
pod/kafka-6ff674c684-5hn8h          1/1     Running   0           3h33m
pod/postgres-58d5565cff-g2qnd       1/1     Running   0           3h33m
pod/rain-processor-597964d56d-tf7pr 1/1     Running   0           3h17m
pod/weather-station-0                1/1     Running   0           3h17m
pod/weather-station-1                1/1     Running   0           3h17m
pod/weather-station-2                1/1     Running   0           3h17m
pod/weather-station-3                1/1     Running   0           3h17m
pod/weather-station-4                1/1     Running   0           3h17m
pod/weather-station-5                1/1     Running   0           3h17m
pod/weather-station-6                1/1     Running   0           3h17m
pod/weather-station-7                1/1     Running   0           3h17m
pod/weather-station-8                1/1     Running   0           3h17m
pod/weather-station-9                1/1     Running   0           3h17m

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/kafka                       ClusterIP      10.96.193.32 <none>        9092/TCP    3h33m
service/postgres                    ClusterIP      10.96.136.108 <none>        5432/TCP    3h33m
service/weather-station              ClusterIP      None          <none>        8080/TCP    3h19m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/central-station      1/1     1             1           3h17m
deployment.apps/kafka                1/1     1             1           3h33m
deployment.apps/postgres              1/1     1             1           3h33m
deployment.apps/rain-processor        1/1     1             1           3h17m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/central-station-845c95bb9 1         1         1       3h17m
replicaset.apps/kafka-6ff674c684          1         1         1       3h33m
replicaset.apps/postgres-58d5565cff        1         1         1       3h33m
replicaset.apps/rain-processor-597964d56d 1         1         1       3h17m

NAME                                READY   AGE
statefulset.apps/weather-station      10/10   3h17m
(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system %

```

Figure 14: Kubernetes Services and the Weather Station StatefulSet.

```

(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather get configmap weather-config -o yaml
apiVersion: v1
data:
  BATCH_SIZE: "5000"
  DB_URL: jdbc:postgresql://postgres:5432/weather
  DROP_RATE: "0.10"
  EMIT_INTERVAL_MS: "1000"
  FLUSH_INTERVAL_MS: "10000"
  KAFKA_BOOTSTRAP_SERVERS: kafka:9092
  TOPIC_RAIN: rain-alerts
  TOPIC_WEATHER: weather-readings
kind: ConfigMap
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"BATCH_SIZE":"5000","DB_URL":"jdbc:postgresql://postgres:5432/weather","DROP_RATE":"0.10","EMIT_INTERVAL_MS":"1000","FLUSH_INTERVAL_MS":"10000","KAFKA_BOOTSTRAP_SERVERS":"kafka:9092","TOPIC_RAIN":"rain-alerts","TOPIC_WEATHER":"weather-readings"},"kind":"ConfigMap","metadata":{"annotations":{},"name":"weather-config","namespace":"weather"}}
  creationTimestamp: "2026-08-23T14:48:30Z"
  name: weather-config
  namespace: weather
  resourceVersion: "1708"
  uid: e74e5a52-0b2a-4c31-98d2-ee4b12153417
(base) ahmedwala@ahmeds-air distributed-weather-monitoring-system %

```

Figure 15: Kubernetes ConfigMap and Secret configuration.

8.6 Runtime Evidence

The following screenshots demonstrate that the same application behavior observed locally is preserved inside Kubernetes: weather station messages are generated, humidity values above 70% produce rain alerts, the central station performs JDBC batch inserts, and SQL queries can be executed against the database from the cluster.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather logs weather-station-0 --tail=15
[station-1] sent=10290 dropped=1187 last={"station_id":1,"s_no":11427,"battery_status":"medium","status_timestamp":1787508930,"weather":{"humidity":87,"temperature":96,"wind_speed":34}}
[station-1] dropped s_no=11448 (total dropped=1148)
[station-1] sent=10320 dropped=1143 last={"station_id":1,"s_no":11463,"battery_status":"medium","status_timestamp":1787508966,"weather":{"humidity":40,"temperature":66,"wind_speed":33}}
[station-1] sent=10350 dropped=1146 last={"station_id":1,"s_no":11496,"battery_status":"medium","status_timestamp":1787508999,"weather":{"humidity":62,"temperature":52,"wind_speed":37}}
[station-1] dropped s_no=11523 (total dropped=1150)
[station-1] sent=10380 dropped=1152 last={"station_id":1,"s_no":11532,"battery_status":"low","status_timestamp":1787509038,"weather":{"humidity":44,"temperature":85,"wind_speed":28}}
[station-2] sent=10410 dropped=1157 last={"station_id":1,"s_no":11567,"battery_status":"medium","status_timestamp":1787509073,"weather":{"humidity":36,"temperature":63,"wind_speed":41}}
[station-1] sent=10440 dropped=1157 last={"station_id":1,"s_no":11597,"battery_status":"high","status_timestamp":1787509183,"weather":{"humidity":90,"temperature":112,"wind_speed":22}}
[station-1] sent=10470 dropped=1159 last={"station_id":1,"s_no":11629,"battery_status":"medium","status_timestamp":1787509135,"weather":{"humidity":19,"temperature":28,"wind_speed":128}}
[station-1] dropped s_no=11634 (total dropped=1160)
[station-1] sent=10500 dropped=1161 last={"station_id":1,"s_no":11661,"battery_status":"low","status_timestamp":1787509167,"weather":{"humidity":91,"temperature":77,"wind_speed":0}}
[station-1] sent=10530 dropped=1163 last={"station_id":1,"s_no":11693,"battery_status":"high","status_timestamp":1787509200,"weather":{"humidity":59,"temperature":50,"wind_speed":77}}
[station-1] sent=10560 dropped=1163 last={"station_id":1,"s_no":11723,"battery_status":"high","status_timestamp":1787509230,"weather":{"humidity":69,"temperature":22,"wind_speed":141}}
[station-1] sent=10590 dropped=1166 last={"station_id":1,"s_no":11756,"battery_status":"low","status_timestamp":1787509263,"weather":{"humidity":12,"temperature":110,"wind_speed":38}}
[station-1] dropped s_no=11774 (total dropped=1170)
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 16: Weather Station pod logs from weather-station-0.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather logs deploy/rain-processor --tail=20
[rain-processor] RAIN station=5 s_no=11731 humidity=73% (alert #3172)
[rain-processor] RAIN station=3 s_no=11730 humidity=96% (alert #3186)
[rain-processor] RAIN station=4 s_no=11732 humidity=80% (alert #3113)
[rain-processor] RAIN station=1 s_no=11731 humidity=86% (alert #3080)
[rain-processor] RAIN station=6 s_no=11731 humidity=94% (alert #3111)
[rain-processor] RAIN station=10 s_no=11734 humidity=86% (alert #3128)
[rain-processor] RAIN station=2 s_no=11731 humidity=77% (alert #3195)
[rain-processor] RAIN station=9 s_no=11735 humidity=73% (alert #3041)
[rain-processor] RAIN station=3 s_no=11733 humidity=98% (alert #3187)
[rain-processor] RAIN station=1 s_no=11734 humidity=71% (alert #3081)
[rain-processor] RAIN station=5 s_no=11735 humidity=71% (alert #3173)
[rain-processor] RAIN station=2 s_no=11734 humidity=75% (alert #3196)
[rain-processor] RAIN station=4 s_no=11737 humidity=77% (alert #3114)
[rain-processor] RAIN station=6 s_no=11736 humidity=85% (alert #3112)
[rain-processor] RAIN station=2 s_no=11736 humidity=96% (alert #3197)
[rain-processor] RAIN station=7 s_no=11737 humidity=87% (alert #3080)
[rain-processor] RAIN station=1 s_no=11739 humidity=91% (alert #3082)
[rain-processor] RAIN station=3 s_no=11739 humidity=100% (alert #3188)
[rain-processor] RAIN station=8 s_no=11740 humidity=96% (alert #3176)
[rain-processor] RAIN station=4 s_no=11741 humidity=72% (alert #3115)
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 17: Rain Processor running in Kubernetes and detecting humidity greater than 70%.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather logs deploy/central-station --tail=20
[central] persisted 1 rain alerts
[central] persisted 2 rain alerts
[central] persisted 1 rain alerts
[central] batch flushed: 94 rows (total 105300)
[central] persisted 5 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 4 rain alerts
[central] persisted 2 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 4 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 1 rain alerts
[central] persisted 2 rain alerts
[central] batch flushed: 96 rows (total 105396)
[central] persisted 5 rain alerts
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system %
```

Figure 18: Central Station logs showing batch insert activity inside Kubernetes.

```
(base) ahmedwalaa@ahmeds-air distributed-weather-monitoring-system % kubectl -n weather exec deploy/postgres -- psql -U weather -d weather -c "SELECT station_id, COUNT(*) AS received, MAX(sequence_number) AS expected FROM weather_readings GROUP BY station_id ORDER BY station_id;"
station_id | received | expected
-----
1 | 10730 | 11909
2 | 10707 | 11908
3 | 10744 | 11909
4 | 10720 | 11911
5 | 10734 | 11910
6 | 10686 | 11908
7 | 10729 | 11910
8 | 10704 | 11910
9 | 10720 | 11910
10 | 10739 | 11911
(10 rows)
```

Figure 19: Database query results executed against the Kubernetes deployment.

9 Cloud Deployment - Bonus

9.1 Target Topology

The bonus deployment removes Kubernetes and distributes the system across two cloud virtual machines plus a managed SQL database. This satisfies the requirement that the Weather Stations and the Central Base Station do not run on the same machine.

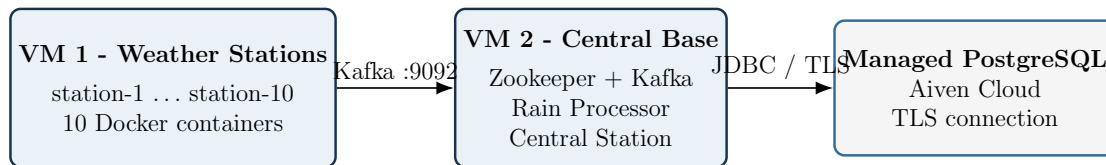


Figure 20: Bonus cloud deployment topology.

9.2 Machine and Provider Details

The repository contains a complete deployment procedure, but the actual provider, instance sizes, IP addresses, and managed database endpoint must be replaced with the values used during the final deployment before submission.

Table 4: Cloud deployment details - replace placeholders with actual deployment values.

Role	Provider	Size	vCPU/RAM	OS / Endpoint
Stations VM	[Provider]	[Size]	[vCPU/RAM]	Ubuntu [version], [IP]
Central VM	[Provider]	[Size]	[vCPU/RAM]	Ubuntu [version], [IP]
Database	Aiven Cloud	[Plan]	Managed	PostgreSQL [version], [host:port]

9.3 Network Configuration

The intended network policy follows least privilege. SSH port 22 is restricted to the administrator address. Kafka port 9092 on the Central VM is reachable only from the Weather Stations VM. The managed PostgreSQL service permits database access only from the required VM addresses. The central station connects using TLS with `DB_SSLMODE=require`.

A key cloud-specific Kafka setting is `advertised.listeners`. The broker advertises an address that the remote Weather Stations VM can actually reach; otherwise the producer may contact the bootstrap address successfully but fail after Kafka metadata discovery redirects it to an unreachable internal hostname.

9.4 Deployment Procedure

The Central VM builds and runs Kafka, Zookeeper, the Rain Processor, and the Central Station through `cloud/docker-compose.central.yml`. Database parameters are loaded from a restricted `central.env` file with mode 600 rather than being committed to the repository.

The Weather Stations VM builds the weather station image, generates a Compose file containing ten station services with distinct IDs, and points all stations to the remote Kafka broker on the Central VM.

Listing 5: Representative Central VM deployment commands.

```
1 sudo apt update
2 sudo apt install -y docker.io docker-compose-plugin git
3
4 git clone <repository-url>
5 cd distributed-weather-monitoring-system
6
7 docker build -t rain-processor:1.0.0 -f rain-processor/Dockerfile .
8 docker build -t central-station:1.0.0 -f central-station/Dockerfile .
9
10 cp cloud/central.env.example central.env
11 chmod 600 central.env
12 # Fill CENTRAL_VM_IP and managed DB credentials.
13
14 docker compose --env-file central.env \
15   -f cloud/docker-compose.central.yml up -d
```

Listing 6: Representative Weather Stations VM deployment commands.

```
1 sudo apt update
2 sudo apt install -y docker.io docker-compose-plugin git
3
4 git clone <repository-url>
5 cd distributed-weather-monitoring-system
6
7 docker build -t weather-station:1.0.0 -f weather-station/Dockerfile .
8 export KAFKA_BOOTSTRAP_SERVERS=<CENTRAL_VM_IP>:9092
9 ./cloud/stations-compose.sh 10 "$KAFKA_BOOTSTRAP_SERVERS"
10 docker compose -f docker-compose.stations.yml up -d
```

10 Challenges and Lessons Learned

The implementation highlights several practical issues that are common in distributed systems:

- **Random simulation must remain measurable.** Incrementing `s.no` before the drop decision creates an observable ground truth for dropped-message analysis.
- **Partition keys affect ordering.** Using `station_id` as the Kafka key preserves order for readings from the same station.
- **Persistence and offset commits must be coordinated.** Committing the database before Kafka offsets prevents acknowledged data from being lost, while database uniqueness constraints absorb replayed records.
- **Batching trades latency for throughput.** A 5000-record threshold is combined with a time-based flush so the system remains responsive during low traffic while still reducing I/O under higher load.

- **Stable identity matters in orchestration.** A StatefulSet provides deterministic station identities without duplicating Kubernetes manifests.
- **Secrets should be externalized.** Environment variables, Kubernetes Secrets, and restricted cloud environment files keep sensitive credentials out of application source code.
- **Cloud Kafka requires correct listener advertisement.** Remote clients must receive a broker address that is routable from their machine after Kafka metadata discovery.

11 Conclusion

This project successfully implements a complete distributed weather monitoring pipeline for the Net Centric Computing lab. Ten mock weather stations continuously produce structured readings, intentional message drops are measurable through sequence gaps, Kafka provides the communication backbone, and a stateful Kafka Streams processor generates rain alerts when humidity exceeds 70%.

The central station persists readings efficiently using batched PostgreSQL writes and manual Kafka offset commits. Database constraints provide idempotent recovery behavior, while SQL analysis validates battery distribution, message-drop rate, latest station state, and rain-alert consistency. The application is containerized and deployed using Kubernetes with stable weather-station identities, persistent database storage, centralized configuration, and separate secret management. The cloud bonus architecture further demonstrates how the same services can be split across independent machines and connected to a managed PostgreSQL service.

Overall, the lab demonstrates the main concerns of a real distributed streaming system: asynchronous messaging, partitioning and ordering, stream processing, persistence, fault-aware delivery semantics, containerization, orchestration, remote networking, and secure configuration management.